

Case Study I: Clustering and Classification

DUE: Monday, Sept 25 @ 11:59 PM

Relevant Textbook Sections: Chapter 4.4 and 4.5

1. Introduction

In this case study you will use the linear algebra and MATLAB skills you have developed to classify handwritten digits.

The main idea of classification is to determine the “meaning” of an unlabeled input. Here, that means determining which digit, 0 through 9, a handwritten image depicts, based only on its vector representation. Classification in this manner is a classical and basic form of so-called “machine learning.” There are many ways to accomplish classification; in this case study you will do it using clustering (see also Chapter 4.4 and 4.5).

You will work with 28x28 images from the MNIST database. Each image is read in as a 784-dimensional vector whose entries range from 0 to 255, indicating levels of gray. In Homework 3 you used MATLAB’s built-in `kmeans()` function as a black box. Here you will build the algorithm yourself, from the concepts of norm and distance. Do not use the built-in `kmeans()` function anywhere in this case study.

2. The provided files

Two skeleton files are provided on Canvas. They are deliberately separate: each time you run the clustering file you generate new centroids, and keeping classification in its own file lets you save a set of centroids from one run and compare it against another.

- `cs1_mnist_base_skeleton.m`, which reads in the data and contains the k-means algorithm for you to complete.
- `cs1_mnist_evaluate_test_set.m`, which evaluates your classifier on the test set.

Both files read images into arrays that are $n \times 785$. Columns 1 through 784 are the pixel values. Column 785 is used to store the centroid assignment for that image, and is initialized to zero.

A training set of 1500 images and a test set of 200 images are provided, along with the correct label for each test image in the array *correctlabels*.

A note on the data. The test set is imperfect by design. A small number of its images are not well-behaved, and you should not expect to classify all 200 correctly no matter how good your clustering is. As a bonus (up to 5%), comment briefly in your report on what you think is going on with the images your classifier gets wrong.

3. Part 1: Implementing k-means

Complete `cs1_mnist_base_skeleton.m`. The `initialize_centroids()` function is written for you. You need to supply:

1. `assign_vector_to_centroid()`, which takes a vector and a set of centroids and returns the index of the closest centroid along with the distance to it.

2. `update_Centroids()`, which recomputes each centroid as the mean of the vectors currently assigned to it.
3. The body of the main for-loop, which repeats assignment and update, and records the k-means cost at each iteration in `cost_iteration`.
4. The cost plot (Figure 1). The centroid montage plot (Figure 2) is already written for you.

You will also need to choose values for k and max_iter at the top of the file.

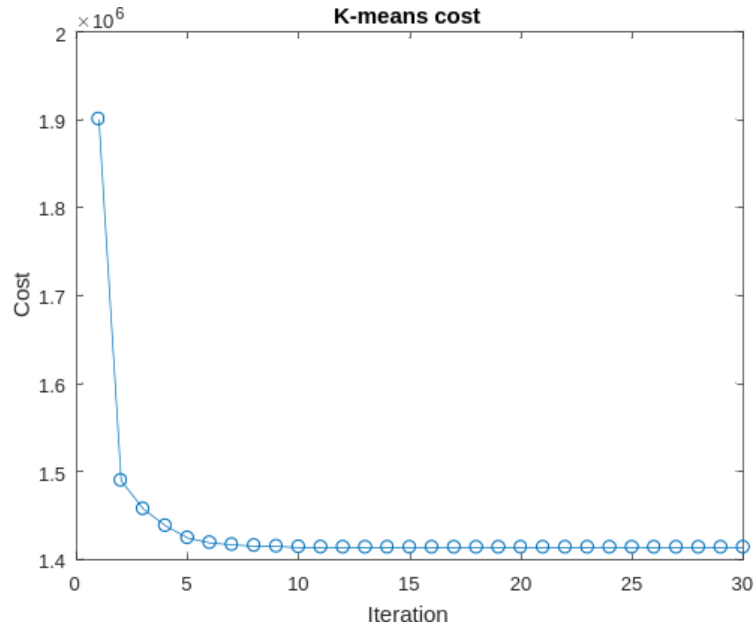


Figure 1. The k-means cost function, as a function of iterations.

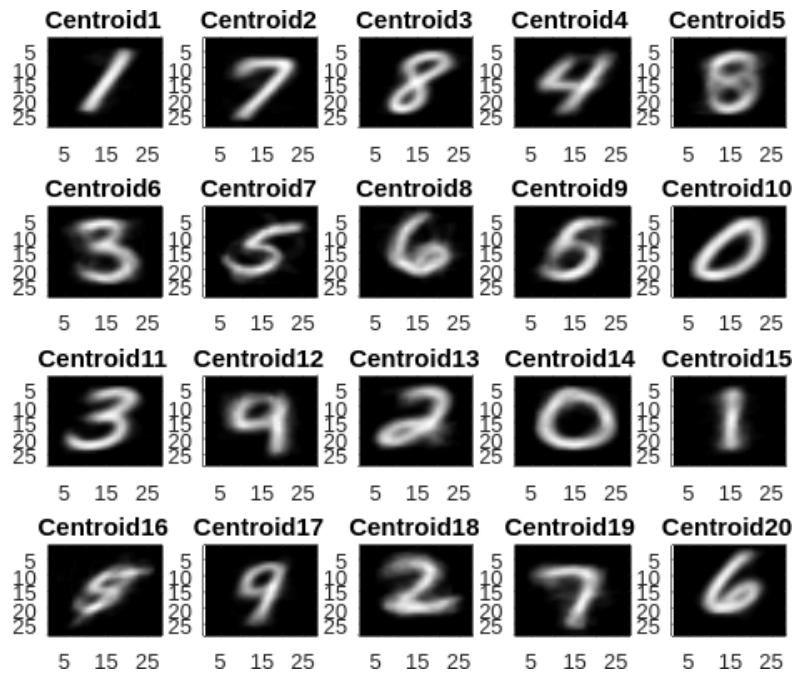


Figure 2. In this example, k was set to 20. The resulting centroids are depicted above.

4. Part 2: Assigning meaning to the centroids

A set of centroids is not yet a classifier: nothing has told MATLAB that a particular centroid means the digit 3. To perform classification, you must assign each centroid a numerical meaning.

The training set labels are available in `trainsetlabels`. How you use them is your decision. Two reasonable strategies are:

- Automatic: for each centroid, look at the true labels of all training images assigned to it and take the most common one.
- Visual adjudication: look at the centroid images from Part 1 and enter the labels yourself. In Figure 2, for example, we would choose (across, then down): 1, 7, 8, 4, 8, 3, 5, 6, 5, 0, 3, 9, 2, 0, 1, 5, 9, 2, 7, 6.

Whichever approach you take, you must be able to justify it in your report.

5. Part 3: Classifying the test set

Once your centroids are labeled, you can classify images the algorithm has never seen. For each test image, use a nearest-neighbor criterion to assign it to a centroid; the label of that centroid is your prediction for that image. A correct classification occurs when the test image's true label matches the numerical interpretation of the centroid to which it is closest.

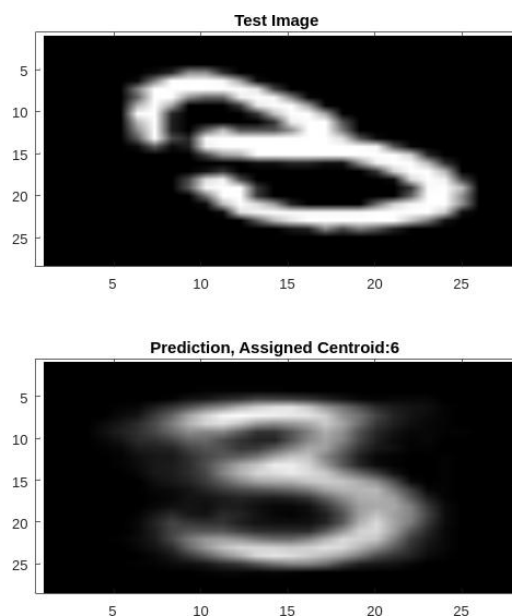


Figure 3. Here, a test image is assigned to Centroid 6 (see the centroids in Figure 2).

Build up `cs1_mnist_evaluate_test_set.m` so that it produces a 200-dimensional vector of predicted labels, reports your overall accuracy against `correctlabels`, and produces a figure similar to Figure 4. Note that the file already contains code that does portions of this.

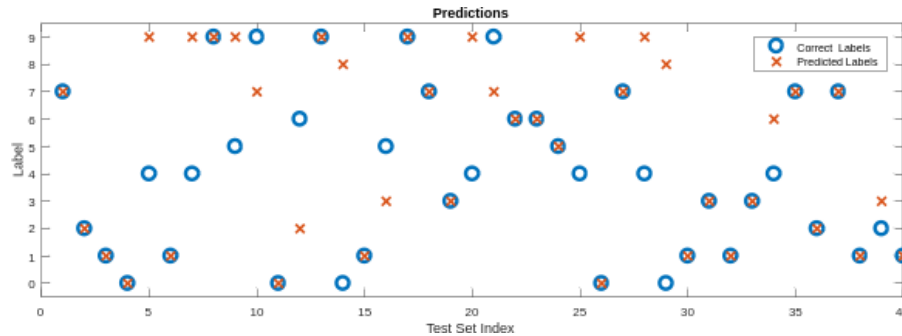


Figure 4. Correct and predicted labels for a 40-image test set. Where the X and O overlay, that test image has been classified correctly. Here, 26/40 are correctly classified. Your test set contains 200 images.

Choosing k

The choice of k affects the accuracy of your results, so structure your code so that you can test different values without rewriting it. Run your classifier for at least three different values of k and record the accuracy each time. You will report these results.

A note on training and testing

An important aspect of classification is the separation between training and testing sets. To test an algorithm, you must use data that were not used in training. For this case study, that means establishing your clusters using one set of data, then measuring performance using a different set.

6. Competition

The `cs1_mnist_evaluate_test_set.m` code written by each group will be used to classify a set of competition images that you have not seen. Performance will be ranked on the accuracy of assigning the correct digit label to each competition image.

7. What to turn in

- Your built-up `cs1_mnist_base_skeleton.m`.
- Your built-up `cs1_mnist_evaluate_test_set.m`, which outputs your predicted labels and reports your accuracy.
- A MAT file named `classifierdata.mat` containing two arrays:
 - A $k \times 785$ array named `centroids` containing the k centroids you wish to use for the competition phase.
 - A $k \times 1$ array named `centroid_labels` containing the numerical meanings of those centroids.
- A two-page report (see below).
- A signed honor code document, as distributed with the case study materials.

You do not need to publish your scripts to PDF for this case study. Your figures belong in the report.

8. The report

Your report should be no more than two pages, including figures. It has three required sections:

- Design. Which value of k did you choose, and why? How did you assign meanings to your centroids, and why that approach?
- Results. What accuracy did your classifier achieve on the test set? Include Figure 1, Figure 2, and Figure 4 from your own run, along with a table or plot of accuracy versus k . Which digits did your classifier confuse most often?
- Conclusions. What do these results tell you about the design? What would you change if you had more time?

You may use the Word or LaTeX template provided with the case study materials, but this is not required. Figures should be clear, with legible axis labels and line widths.

In addition to the report, we will ask a random sampling of students to explain their case study and design choices in a 1-1 interview, outside of class time.

9. Rubric

- Correctness of MATLAB code (40%), including the success rate of classification on the test data and the competition data.
- Study design (30%): motivation and justification for your choice of k and your centroid-labeling strategy.
- Presentation (30%): plots and figures should be clear, with large and legible axis numbers and legible lines; the report should be well organized and clearly written.
- Bonus (up to 5%): your comments on the misclassified test images.

10. Tips

- The skeleton file shows how to reshape a 784-element vector into a 28×28 array that can be plotted as an image. Note the transpose.
- Loops are going to be very useful throughout this case study.
- In MATLAB, functions should either appear at the very end of your script or in an entirely separate file.
- Be careful to keep the pixel columns (1 through 784) separate from the assignment column (785) when computing distances.
- You are free to choose k and the number of iterations, and to adjust the overall logic of the code as needed, but any unique adjustments must be documented in your final submission.